

BANDroom macOS setup and what was done

What I did for you

1. Installed the .NET SDK on your Mac so we can build .NET projects there.
2. Created a new cross-platform solution called `Bandroom.sln`.
3. Added a shared engine library in `src/Bandroom.Core`.
4. Created a Mac desktop app skeleton in `src/Bandroom.Mac` using Avalonia.
5. Wired the Mac app to the shared engine with a demo button.
6. Verified the new solution builds successfully on macOS.

How it works now

The shared brain

- `Bandroom.Core` is the shared trigger engine.
- It has a `GameState` object that holds the current and previous play.
- It calculates the `Delta` between plays so we can tell if yards were gained or lost.
- It uses an `EventRouter` and independent rule helpers.

The Mac app

- `Bandroom.Mac` is a simple Avalonia desktop app for macOS.
- It uses the shared engine from `Bandroom.Core`.
- It has a button called `Run Demo Trigger`.
- When clicked, it runs a sample state through the rule engine and shows the fired events.

Events that are wired in the demo

The example rules currently implemented are:

- `TFL` — Tackle For Loss detection
- `OFF\_ON\_1ST\_DOWN` — offense on 1st down
- `OFF\_ON\_2ND\_DOWN` — offense on 2nd down
- `OFF\_ON\_3RD\_DOWN` — offense on 3rd down
- `2ND\_TIMEOUT\_4\_REMAINING` — 2nd timeout when 4 remain
- `3RD\_TIMEOUT\_3\_REMAINING` — 3rd timeout when 3 remain
- `4TH\_TIMEOUT\_2\_REMAINING` — 4th timeout when 2 remain
- `5TH\_TIMEOUT\_1\_REMAINING` — 5th timeout when 1 remain
- `6TH\_TIMEOUT\_0\_REMAINING` — 6th timeout when 0 remain
- `BIG\_GAIN\_TOUCHDOWN` — offense gain over 10 yards resulting in touchdown
- `DEF\_STOP\_ON\_3RD` — defense stop on 3rd down
- `DEF\_STOP\_ON\_4TH` — defense stop on 4th down
- `DEF\_ON\_3RD\_DOWN\_ONLY\_IF\_LOSS\_PREV\_DOWN` — defense on 3rd down if previous down lost
- `DEF\_ON\_2ND\_DOWN` — defense on 2nd down

How to use it right now

1. Open the project folder on your Mac.
2. Run `dotnet build Bandroom.sln` in Terminal.
3. Run the Mac app by executing `dotnet run --project src/Bandroom.Mac/Bandroom.Mac.csproj`.
4. Click `Run Demo Trigger` in the Mac app window.

5. Read the text output to see which events fired.

What the files do

- `src/Bandroom.Core/GameState.cs` — stores the current and previous play state.
- `src/Bandroom.Core/PlaySnapshot.cs` — holds a single play snapshot.
- `src/Bandroom.Core/PlayDelta.cs` — computes yards gained/lost and possession changes.
- `src/Bandroom.Core/EventRouter.cs` — sends the state to each rule helper.
- `src/Bandroom.Core/IRuleEvaluator.cs` — interface for rule helpers.
- `src/Bandroom.Core/Helpers/\*.cs` — rule helpers for each event.
- `src/Bandroom.Mac/MainWindow.axaml` — Mac UI layout.
- `src/Bandroom.Mac/MainWindow.axaml.cs` — Mac UI behavior and demo wiring.

What we need to do next

1. Add more rules to `Bandroom.Core/Helpers` for every event you asked for.
2. Connect the real OCR engine from the Windows app to the shared `GameState` input.
3. Build a macOS capture adapter if you want live screen-based input on Mac.
4. Add real audio playback in `Bandroom.Mac` for Mac instead of just a demo button.
5. Keep the shared `Bandroom.Core` engine and add a Windows host that also uses it.

Why this is a good setup

- The brain is shared between Windows and Mac.
- We only write the play/rule logic once.
- The Mac app can grow later to use real game-screen input.
- The Windows version can still keep working with the same shared engine.

In 5-year-old terms

Imagine Bandroom is a toy robot. The robot has:

- a brain (`Bandroom.Core`),
- a Mac body (`Bandroom.Mac`),
- and later a Windows body.

The brain decides when to play sounds. The Mac body just shows a button and waits for the brain to say v

I built the robot's brain and put it into a simple Mac body. Now the robot can think on your Mac.